



**KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN MẠNG VÀ CÁC HỆ THỐNG THÔNG TIN**

CHƯƠNG 6

QUẢN LÝ BỘ NHỚ



Nội dung chính - Chương 6

Mục lục

- 6.1. Các loại địa chỉ
- 6.2. Không gian địa chỉ
- 6.3. Swapping & Overlay
- 6.4. Cấp phát liên tục
- 6.5. Phân trang
- 6.6. Phân đoạn

Từ khóa

Address binding	Logical address
Physical address	MMU
Swapping	Fragmentation
Paging	Overlay
Page table	Segmentation

Trọng tâm kiến thức



Hiểu cách địa chỉ được tạo

Phân biệt địa chỉ trong chương trình và vị trí thật trong RAM; hiểu vai trò của MMU.

Biết cách tính địa chỉ

Thực hiện các bước dịch địa chỉ trong cấp phát liên tục, phân trang và phân đoạn.

So sánh các cách quản lý

Nêu được ưu, nhược điểm của cấp phát liên tục, Swapping, Overlay, phân trang và phân đoạn.

Câu hỏi định hướng: Làm thế nào để nhiều chương trình dùng chung RAM an toàn và hiệu quả?



6.1. Các loại địa chỉ

Địa chỉ của chương trình được chuyển đổi qua nhiều bước trước khi trở thành vị trí thật trong RAM.



Vì sao chương trình cần địa chỉ bộ nhớ?

Muốn chạy, chương trình phải được đưa vào RAM và được hệ điều hành tạo thành một tiến trình.

Mỗi câu lệnh và dữ liệu đều cần một địa chỉ để CPU biết nơi đọc hoặc ghi.

Địa chỉ có thể được gán ở một trong ba thời điểm trên.

Khi biên dịch

Khi nạp vào RAM

Khi chương trình đang chạy



Nhiệm vụ của quản lý bộ nhớ

Dễ sử dụng

Chương trình chỉ dùng địa chỉ logic, không cần biết chính xác đang nằm ở đâu trong RAM.

An toàn

Mỗi tiến trình chỉ được đọc và ghi trong vùng nhớ được phép.

Tiết kiệm RAM

Hệ điều hành cố gắng dùng RAM hợp lý và hạn chế phần bộ nhớ bị lãng phí.

Quản lý bộ nhớ giúp chương trình nhìn thấy một vùng nhớ đơn giản, trong khi phần cứng xử lý vị trí thật.



Các dạng địa chỉ qua từng giai đoạn

Địa chỉ ký hiệu

Tên biến hoặc nhãn lệnh trong mã nguồn, ví dụ: count, loop.

Địa chỉ tương đối

Vị trí tính từ đầu chương trình, ví dụ: cách đầu mô-đun 14 byte.

Địa chỉ logic (ảo)

Địa chỉ CPU tạo ra và chương trình sử dụng khi chạy.

Địa chỉ vật lý

Vị trí thật trong RAM sau khi MMU dịch địa chỉ.



Gán địa chỉ khi biên dịch

Cách làm

Biết trước chương trình sẽ nằm ở đâu trong RAM nên trình biên dịch tạo luôn địa chỉ cố định.

Hạn chế

Nếu chuyển chương trình sang vị trí khác, phải biên dịch lại.

Cách này đơn giản nhưng chỉ phù hợp khi vị trí nạp không thay đổi.



Gán địa chỉ khi nạp chương trình

Cách làm

Trình biên dịch tạo địa chỉ tương đối; bộ nạp chọn vị trí trong RAM và cộng thêm địa chỉ bắt đầu.

Khả năng di chuyển

Mỗi lần chạy có thể nạp ở vị trí khác, nhưng thường không di chuyển sau khi đã nạp.

Linh hoạt hơn gán khi biên dịch, nhưng chưa phù hợp với việc di chuyển tiến trình trong lúc chạy.



Gán địa chỉ khi chương trình đang chạy

Cách làm

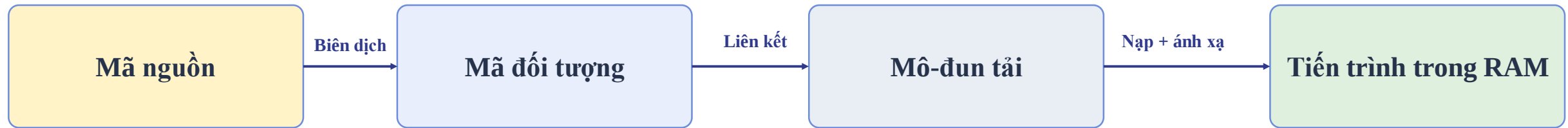
CPU tạo địa chỉ logic; đến lúc truy cập, MMU mới dịch thành địa chỉ vật lý.

Cần phần cứng hỗ trợ

MMU, thanh ghi base/limit, bảng trang hoặc bảng đoạn thực hiện việc dịch và kiểm tra quyền truy cập.

Đây là nền tảng của Swapping, phân trang và bộ nhớ ảo.

Từ địa chỉ ký hiệu đến địa chỉ vật lý



Mã nguồn được biên dịch, liên kết và nạp vào RAM. Khi chạy, MMU tiếp tục đổi địa chỉ logic thành địa chỉ vật lý.



Tải động: Cần hàm nào mới nạp hàm đó

Cách hoạt động

Chương trình chính được nạp trước. Một hàm hoặc thủ tục chỉ được đưa vào RAM khi được gọi.

Lợi ích

Không tốn RAM cho phần mã hiếm khi dùng.

Ai quản lý?

Chương trình hoặc thư viện quyết định phần nào cần nạp; hệ điều hành hỗ trợ đọc tệp và cấp vùng nhớ.

Tải động làm giảm lượng mã phải nằm thường xuyên trong RAM.



Liên kết động: Dùng thư viện khi chương trình chạy

Đoạn mã trung gian (stub)

Lời gọi đi qua một đoạn mã nhỏ để tìm hàm cần dùng trong thư viện; nhiều tiến trình có thể dùng chung mã thư viện.

Vai trò của hệ điều hành

Nạp thư viện, tìm đúng hàm, kiểm tra phiên bản và cập nhật địa chỉ gọi.

Liên kết động giúp tệp chương trình nhỏ hơn, nhưng phải bảo đảm thư viện đúng phiên bản.



Gán địa chỉ càng muộn càng linh hoạt

Biên dịch

Nhanh và đơn giản; vị trí cố định, phải biên dịch lại khi đổi chỗ.

Tải

Mã tái định vị; chọn vị trí khi nạp, khó di chuyển sau đó.

Ghi nhớ: Càng trì hoãn việc quyết định vị trí thật, hệ thống càng linh hoạt nhưng cần phần cứng và dữ liệu quản lý phức tạp hơn.

Ánh xạ động; hỗ trợ di chuyển, chia sẻ và bộ nhớ ảo.

Nguyên tắc

Càng trì hoãn quyết định, hệ thống càng linh hoạt nhưng cần thêm phần cứng và dữ liệu quản lý.



6.2. Không gian địa chỉ

Chương trình nhìn thấy địa chỉ logic; MMU đổi chúng thành vị trí thật trong RAM.



Địa chỉ chương trình thấy và vị trí thật trong RAM

Không gian địa chỉ logic

Tập các địa chỉ mà một tiến trình có thể sử dụng; thường bắt đầu từ 0 và trông như một vùng liên tục.

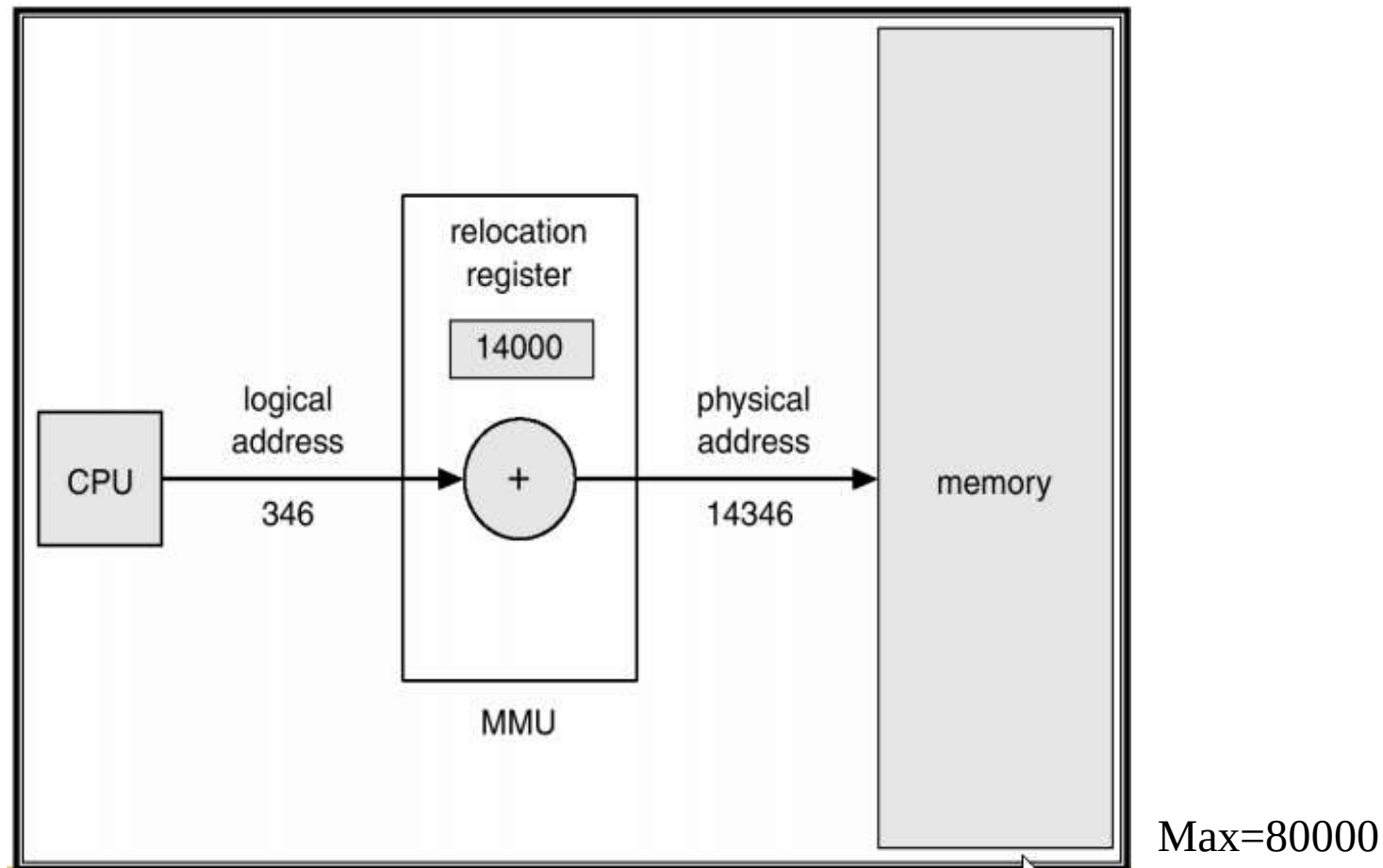
Không gian địa chỉ vật lý

Các vị trí thật trong RAM; dữ liệu của tiến trình có thể nằm ở nhiều nơi khác nhau.

Khi gán địa chỉ lúc chạy, MMU dịch địa chỉ mỗi lần CPU truy cập bộ nhớ.

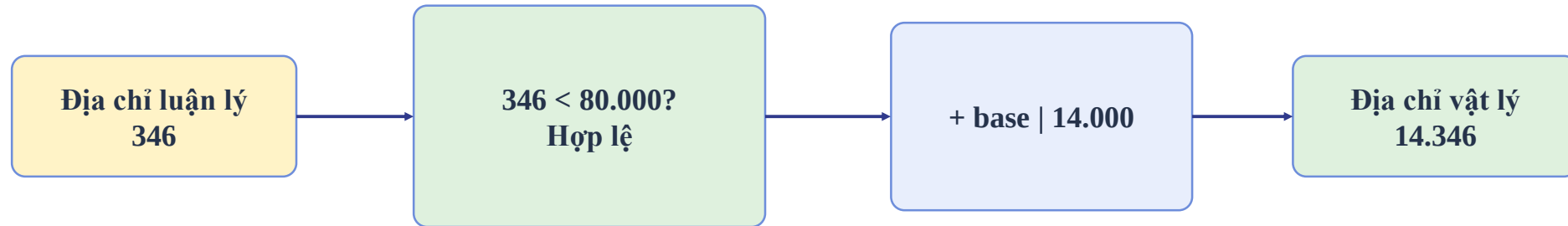


MMU là “bộ phiên dịch” địa chỉ



CPU đưa ra địa chỉ logic. MMU kiểm tra quyền truy cập và chuyển thành địa chỉ vật lý trước khi gửi đến RAM.

Dịch địa chỉ bằng base và limit



Nếu địa chỉ logic nhỏ hơn limit thì hợp lệ. MMU cộng base (relocation) để tìm địa chỉ vật lý; nếu vượt giới hạn, hệ điều hành báo lỗi.



Mỗi tiến trình có vùng nhớ riêng

Tách biệt

Hai tiến trình có thể cùng dùng địa chỉ 0x1000 nhưng được MMU đưa tới hai vị trí RAM khác nhau.

Nhìn như liên tục

Chương trình thấy một vùng nhớ liên mạch dù dữ liệu thật có thể nằm rải rác.

Chia sẻ có kiểm soát

Hệ điều hành có thể cho nhiều tiến trình cùng dùng một khung trang hoặc đoạn dữ liệu.

Không gian địa chỉ vừa bảo vệ các tiến trình, vừa giúp chương trình dễ làm việc với bộ nhớ.



Tách địa chỉ logic và vật lý có lợi gì?

Dễ di chuyển

Tiến trình có thể được đặt ở vị trí khác mà không phải sửa từng địa chỉ trong chương trình.

Bảo vệ

MMU kiểm tra vùng hợp lệ và quyền đọc, ghi, thực thi.

Chia sẻ

Nhiều tiến trình có thể dùng chung thư viện hoặc vùng dữ liệu được cho phép.

Mở rộng

Không gian ảo có thể lớn hơn RAM; phần chưa dùng chưa cần nạp vào bộ nhớ.



6.3. Swapping & Overlay

Swapping tạm đưa tiến trình ra ổ đĩa để lấy chỗ trống trong RAM.

Overlay chia chương trình thành các phần thay thế nhau và chỉ nạp phần đang cần.



Swapping: tạm đưa tiến trình ra khỏi RAM

Swap out – đưa ra

Mã và dữ liệu của tiến trình được ghi ra vùng swap; phần RAM đang chiếm được trả lại cho hệ thống.

Swap in – đưa vào

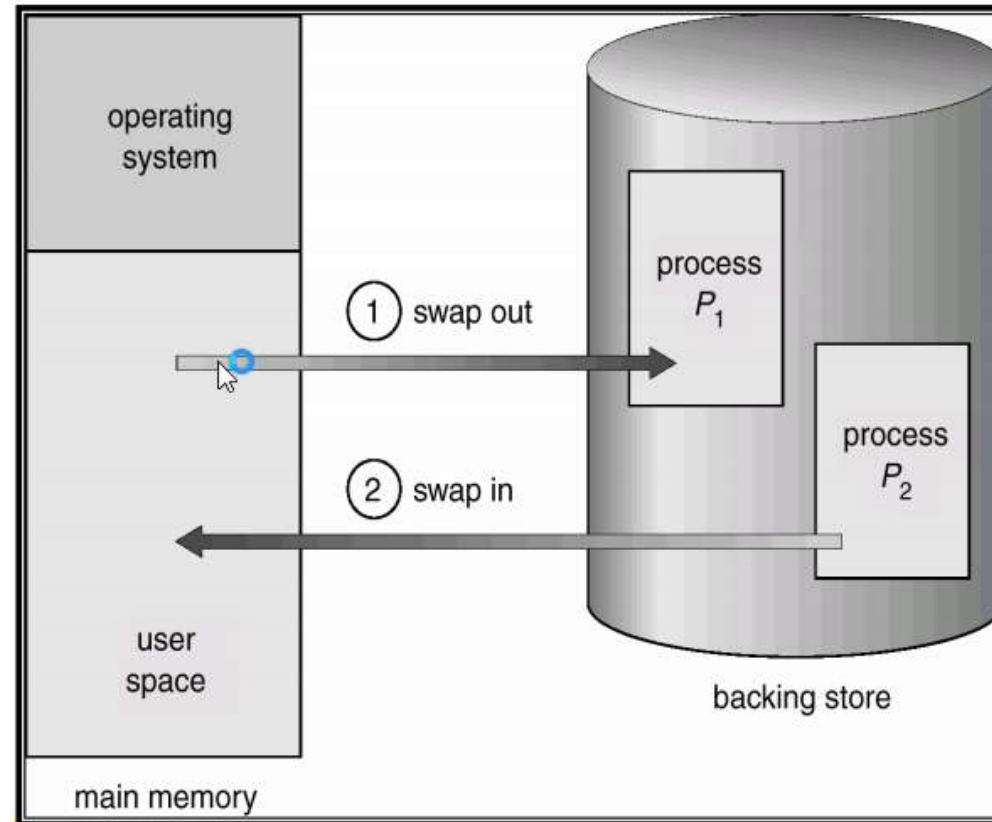
Khi cần chạy tiếp, tiến trình được đọc từ vùng swap vào RAM và khôi phục ảnh xạ.

Điều kiện

Ổ đĩa phải có vùng swap đủ lớn. Tốc độ đọc/ghi ảnh hưởng trực tiếp đến thời gian chờ.

Tiến trình ở ngoài RAM chưa thể chạy, dù trạng thái của nó vẫn có thể là sẵn sàng hoặc đang chờ.

Sơ đồ Swapping



Hệ điều hành lưu trạng thái, ghi tiến trình ra vùng swap và thu hồi RAM. Khi cần, các bước được thực hiện theo chiều ngược lại.



Quá trình Swapping

1. Chọn tiến trình

Thường chọn tiến trình đang chờ, ít thay đổi dữ liệu và không giữ tài nguyên khó di chuyển.

2. Lưu trạng thái

Tạm dừng tiến trình, lưu thanh ghi và bảo đảm thao tác vào/ra không còn dùng vùng nhớ cũ.

3. Chuyển dữ liệu

Ghi mã và dữ liệu ra vùng swap, sau đó đánh dấu tiến trình đang ở ngoài RAM.

4. Nạp lại

Cấp vùng RAM, đọc dữ liệu vào, cập nhật địa chỉ và đưa tiến trình trở lại hàng đợi sẵn sàng.



Roll out / roll in: nhường RAM cho tiến trình ưu tiên cao

Roll out – đưa ra

Khi thiếu RAM, tiến trình ưu tiên thấp có thể tạm bị đưa ra vùng swap.

Roll in – đưa vào

Tiến trình ưu tiên cao được nạp vào RAM và chạy trước; tiến trình cũ được nạp lại sau.

Cần có cơ chế chống để tiến trình ưu tiên thấp phải chờ quá lâu.



Thời gian Swapping phụ thuộc lượng dữ liệu

Ví dụ

Tiến trình 200 MB, tốc độ đọc/ghi 100 MB/s:
đưa ra mất khoảng 2 giây; đưa ra rồi nạp lại
mất khoảng 4 giây.

Chi phí khác

Còn có thời gian chờ thiết bị, ghi dữ liệu đã
thay đổi và cập nhật các bảng quản lý.

Swapping chậm hơn truy cập RAM rất nhiều, vì vậy chỉ nên dùng khi thật sự cần giải phóng bộ nhớ.



Đang vào/ra dữ liệu thì không nên di chuyển vùng nhớ

Vì sao?

Thiết bị có thể đang đọc hoặc ghi trực tiếp vào vùng nhớ của tiến trình. Di chuyển vùng này có thể làm dữ liệu đi sai chỗ.

Ghim trang (pinning)

Hệ điều hành tạm khóa các trang đang dùng cho I/O, không cho swap hoặc di chuyển đến khi thao tác kết thúc.

Dùng bộ đệm của kernel

Thiết bị trao đổi với vùng nhớ của kernel trước, sau đó dữ liệu mới được sao chép sang tiến trình.

An toàn dữ liệu quan trọng hơn việc thu hồi RAM sớm.



Phân biệt Swapping với các kỹ thuật gần giống

Chuyển ngữ cảnh

Chỉ đổi tiến trình đang dùng CPU; các tiến trình vẫn có thể cùng nằm trong RAM.

Swapping

Đưa toàn bộ hoặc phần lớn tiến trình giữa RAM và vùng swap để lấy chỗ trống.

Phân trang

Quản lý theo từng trang nhỏ; một số trang ở RAM, số khác có thể nằm ngoài RAM.

Ngủ đông

Lưu trạng thái của cả hệ thống để tắt máy và khôi phục sau.



Hệ thống hiện đại thường đổi theo từng trang

Đơn vị nhỏ hơn

Thay vì đưa cả tiến trình ra ngoài, hệ điều hành chỉ chọn các trang ít dùng để thu hồi khung RAM.

Khi thiếu RAM kéo dài

Máy có thể phải đọc/ghi ổ đĩa liên tục và chạy chậm rõ rệt.

Mục tiêu

Giữ các trang đang dùng nhiều trong RAM và đưa các trang ít dùng ra ngoài.

Swapping toàn tiến trình là khái niệm nền tảng; hệ thống hiện đại chủ yếu kết hợp swapping với phân trang và bộ nhớ ảo.

Overlay là gì?



Mục tiêu

Cho phép chương trình lớn hơn phần RAM dành cho nó vẫn có thể chạy.

Cách làm

Chia chương trình thành phần luôn cần và các mô-đun không dùng đồng thời.

Nguyên tắc

Chỉ nạp mô-đun đang cần; mô-đun mới ghi đè lên vùng của mô-đun cũ.

**Overlay tập trung vào các phần của một chương trình;
Swapping tập trung vào tiến trình hoặc các trang bộ nhớ.**



Cấu trúc bộ nhớ khi dùng Overlay

Chương trình được chia thành phần cố định và một vùng dùng chung cho các mô-đun thay thế nhau.

Các mô-đun của chương trình

Phần gốc
20 KB

Mô-đun A
30 KB

Mô-đun B
50 KB



Vùng RAM thực sự cần

Phần gốc
20 KB

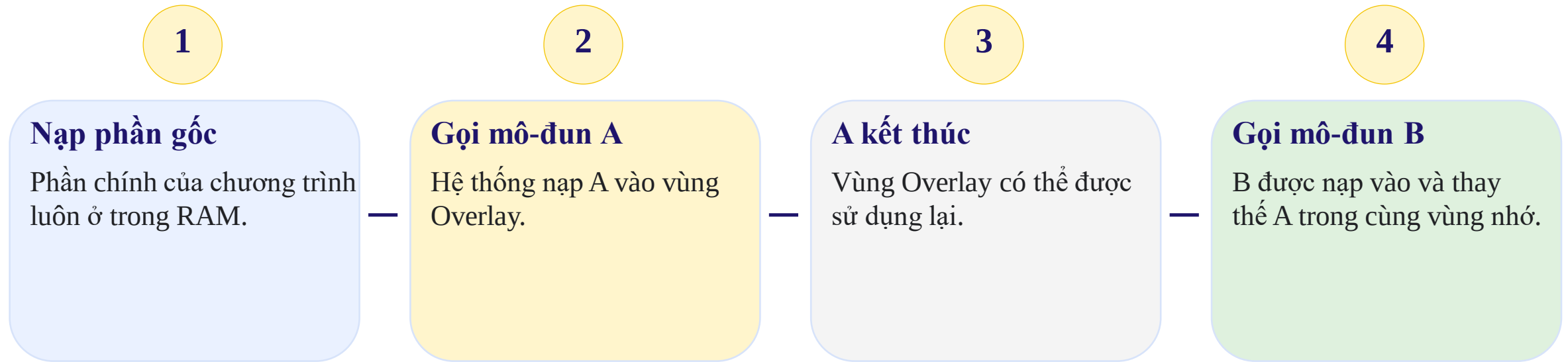
Vùng Overlay
50 KB

Nạp A hoặc B

Dung lượng cần = phần gốc + mô-đun lớn nhất = 20 + 50 = 70 KB, thay vì 100 KB.



Overlay hoạt động theo từng bước



Điểm quan trọng: A và B không cần có mặt trong RAM cùng lúc.



Ví dụ tính dung lượng bộ nhớ khi dùng Overlay

Giả sử chương trình gồm:

Phần gốc: 20 KB

Mô-đun A: 30 KB

Mô-đun B: 50 KB

Không dùng Overlay

Phải nạp cả ba phần:
 $20 + 30 + 50 = 100 \text{ KB}$

Có dùng Overlay

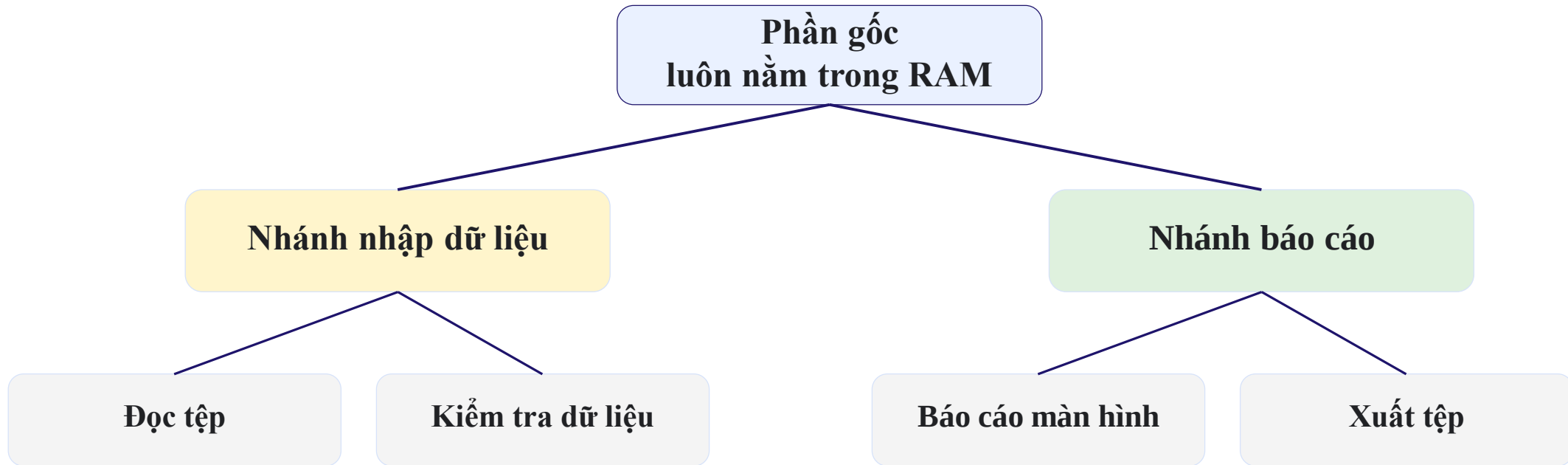
Chỉ cần phần gốc và
mô-đun lớn nhất:
 $20 + \max(30, 50) = 70 \text{ KB}$

Tiết kiệm 30 KB vì A và B dùng chung một vùng nhớ và không chạy đồng thời.



Cây Overlay mô tả các phần có thể thay thế nhau

Các mô-đun cùng nhánh cần theo quan hệ gọi; các mô-đun ở hai nhánh không dùng đồng thời có thể dùng chung vùng nhớ.



**Thiết kế cây Overlay phải dựa trên luồng gọi chương trình:
Phần nào không cần cùng lúc mới được dùng chung vùng nhớ.**

Ưu điểm và hạn chế của Overlay



Ưu điểm

- Chạy được chương trình lớn trong RAM nhỏ.
- Giảm lượng mã phải nằm đồng thời trong bộ nhớ.
- Phù hợp với hệ thống nhúng hoặc thiết bị có RAM hạn chế.

Hạn chế

- Phải chia chương trình thành các mô-đun hợp lý.
- Nạp mô-đun từ ổ đĩa làm chương trình chậm hơn.
- Lập trình và kiểm thử phức tạp hơn.

Ngày nay bộ nhớ ảo và phân trang đã thay thế phần lớn Overlay, nhưng Overlay vẫn quan trọng về mặt nguyên lý và còn dùng trong một số hệ nhúng.



Swapping và Overlay khác nhau như thế nào?

Swapping

- Đơn vị: toàn bộ tiến trình hoặc nhiều trang.
- Người quản lý: hệ điều hành.
- Mục tiêu: giải phóng RAM cho tiến trình khác.
- Dữ liệu di chuyển giữa RAM và vùng swap.

Overlay

- Đơn vị: các mô-đun trong cùng chương trình.
- Người quản lý: chương trình, bộ nạp hoặc trình liên kết.
- Mục tiêu: chạy chương trình lớn trong vùng RAM nhỏ.
- Mô-đun mới thay thế mô-đun cũ trong cùng vùng nhớ.

Ghi nhớ: Swapping đổi tiến trình; Overlay đổi phần chương trình.



6.4. Cấp phát liên tục

Mỗi tiến trình chiếm một vùng RAM liên nhau. Hệ điều hành phải chọn vùng trống phù hợp và hạn chế lãng phí.



RAM được chia cho hệ điều hành và các tiến trình

Vùng hệ điều hành

Chứa nhân hệ điều hành, chương trình xử lý ngắt và các dữ liệu quản lý quan trọng.

Vùng tiến trình

Mỗi tiến trình nhận một dải RAM liên nhau; base là địa chỉ bắt đầu, limit là kích thước tối đa.

Hệ điều hành theo dõi

Danh sách vùng đã dùng và vùng còn trống để cấp phát hoặc thu hồi.

Ưu điểm: Dịch địa chỉ đơn giản. Nhược điểm: Khó tận dụng các vùng trống nằm rải rác.



Hai cách chia vùng nhớ liên tục

Phân vùng cố định

RAM được chia sẵn thành các vùng. Dễ quản lý nhưng dễ lãng phí phần dư bên trong và giới hạn số tiến trình.

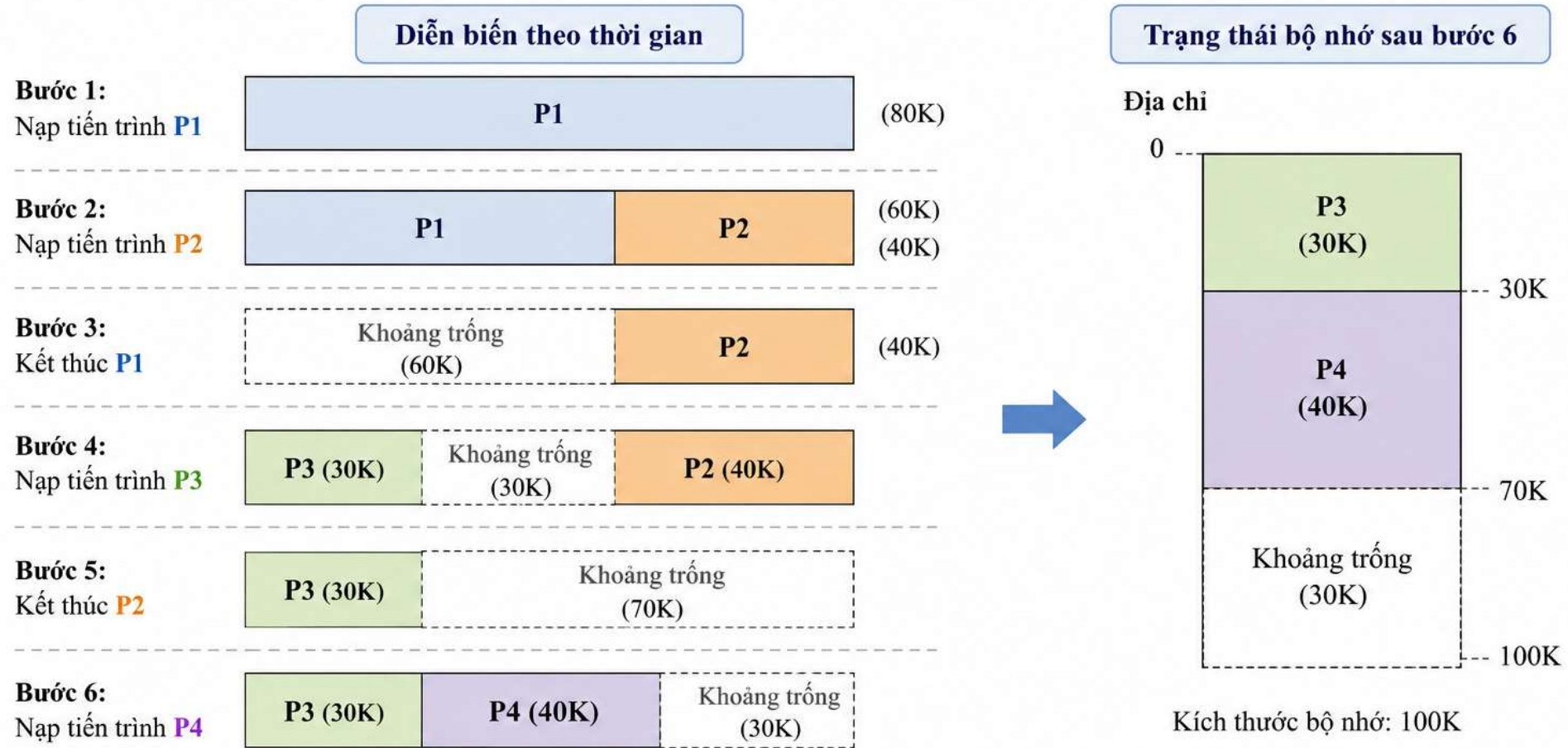
Phân vùng động

Mỗi tiến trình được cấp đúng một vùng theo kích thước cần dùng; lâu dần tạo nhiều khoảng trống nhỏ.

Điểm chung: toàn bộ tiến trình vẫn phải nằm trong một vùng RAM liền nhau.



Phân vùng động tạo các khoảng trống xen kẽ



Khi tiến trình kết thúc, vùng của nó trở thành khoảng trống. Chỉ các khoảng trống nằm cạnh nhau mới ghép lại được.



Chọn khoảng trống nào để nạp tiến trình?

Đầu vào

Kích thước tiến trình và danh sách các vùng RAM đang trống.

Cách chọn

Có thể dùng First-fit, Best-fit hoặc Worst-fit; phần còn dư tiếp tục được ghi vào danh sách trống.

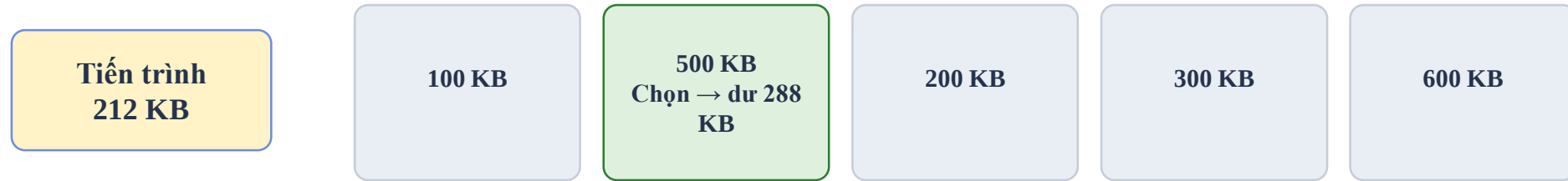
Mục tiêu

Tìm nhanh, giảm phần dư quá nhỏ và giữ đủ vùng trống cho các yêu cầu sau.

Không có thuật toán tốt nhất cho mọi trường hợp; kết quả phụ thuộc thứ tự cấp phát và giải phóng.



First-fit dừng ở khoảng trống đầu tiên đủ lớn

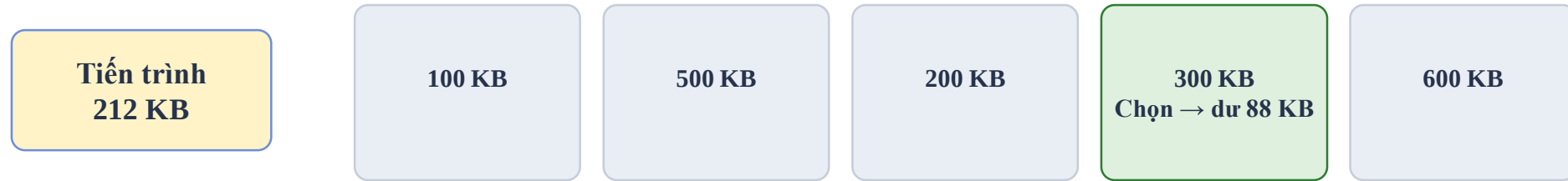


First-fit: Khoảng trống đầu tiên đủ lớn

Với các khoảng trống 100, 500, 200, 300, 600 KB và yêu cầu 212 KB, First-fit chọn khoảng trống 500 KB; phần còn lại là 288 KB.



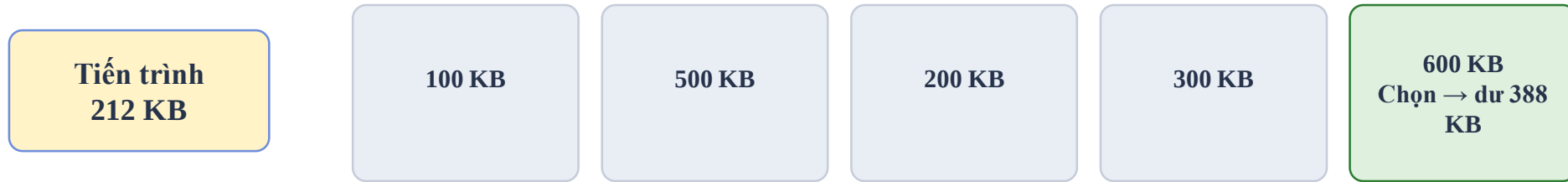
Best-fit chọn khoảng trống đủ nhỏ nhất



Best-fit: Khoảng trống nhỏ nhất nhưng vẫn đủ

Best-fit quét danh sách và chọn khoảng trống 300 KB cho yêu cầu 212 KB; phần dư 88 KB là nhỏ nhất trong các lựa chọn hợp lệ.

Worst-fit lấy khoảng trống lớn nhất



Worst-fit: Khoảng trống lớn nhất

Worst-fit chọn khoảng trống 600 KB cho yêu cầu 212 KB và để lại 388 KB; hy vọng phần dư vẫn hữu ích cho yêu cầu sau.



So sánh nhanh First-fit, Best-fit và Worst-fit

First-fit

Chọn vùng trống đầu tiên đủ lớn; thường tìm nhanh.

Best-fit

Chọn vùng vừa khít nhất; phần dư nhỏ nhưng dễ tạo nhiều mảnh vụn.

Worst-fit

Chọn vùng lớn nhất; để lại phần dư lớn nhưng thường phải tìm toàn bộ danh sách.

Ghi nhớ

First-fit thường đơn giản và nhanh; kết quả thực tế còn phụ thuộc chuỗi yêu cầu.

Còn nhiều RAM trống nhưng vẫn không cấp được



Biểu hiện

Tổng RAM trống đủ lớn, nhưng bị chia thành nhiều vùng nhỏ nên không có một vùng liên nhau đủ chứa tiến trình.

Nguyên nhân

Các lần cấp và trả bộ nhớ làm vùng đã dùng và vùng trống xen kẽ.

Cách xử lý



Phân mảnh trong: Phần RAM dư nằm bên trong vùng đã cấp

Biểu hiện

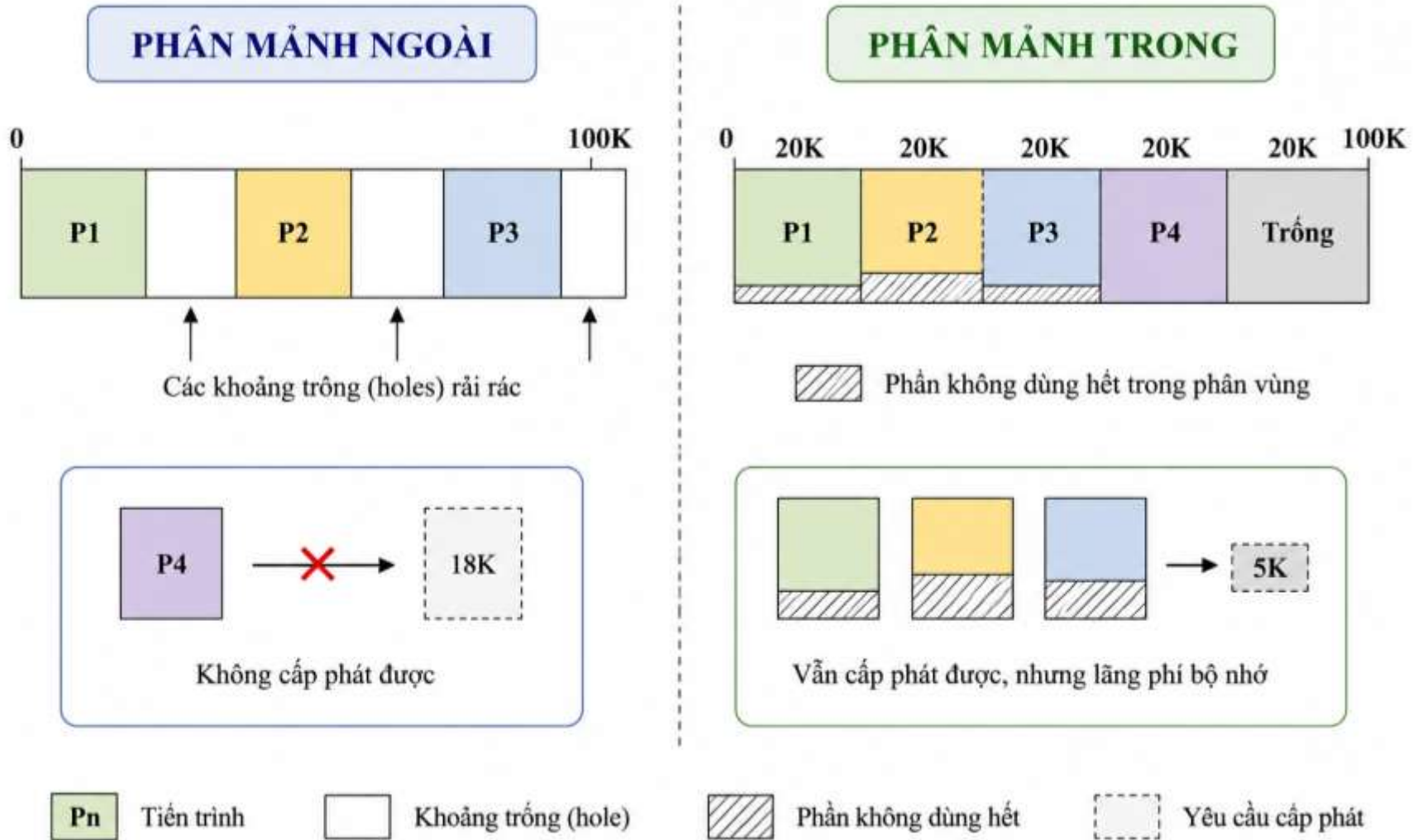
Hệ thống cấp theo khối cố định nên đôi khi cấp nhiều hơn nhu cầu thật; phần thừa không dùng cho tiến trình khác được.

Ví dụ

Tiến trình cần 70 KB nhưng nhận vùng 100 KB: 30 KB còn lại là phân mảnh trong.

Phân vùng cố định và phân trang đều có phân mảnh trong; kích thước đơn vị càng lớn thì lãng phí tiềm năng càng cao.

Phân mảnh ngoài và trong khác vị trí



Phân mảnh ngoài nằm giữa các vùng đã cấp; phân mảnh trong nằm ngay bên trong vùng đã cấp.



Gom các vùng trống thành một khối lớn



Hệ điều hành dời các tiến trình về một phía. Tổng RAM trống không đổi nhưng trở thành một vùng liền nhau để cấp phát hơn.



Gom vùng trống cần cho phép di chuyển tiến trình

Điều kiện

Địa chỉ phải được gán lúc chạy để MMU cập nhật vị trí mới mà không phải sửa chương trình.

Chi phí

Phải sao chép nhiều dữ liệu, tạm dừng tiến trình và cập nhật thông tin quản lý.

Giới hạn

Một số vùng đang dùng cho I/O, vùng chia sẻ hoặc địa chỉ vật lý cố định không thể di chuyển.

Vì gom vùng trống tốn thời gian, hệ thống thường dùng phân trang để tránh yêu cầu vùng RAM liên tục.



Bài tập: Cấp lần lượt 212 KB, 417 KB và 112 KB

Danh sách khoảng trống ban đầu

100 KB, 500 KB, 200 KB, 300 KB, 600 KB.
Hãy mô phỏng First-fit và ghi lại danh sách khoảng trống sau mỗi yêu cầu.

So sánh

Lập lại bằng Best-fit. Thuật toán nào cấp được yêu cầu 426 KB tiếp theo? Giải thích dựa trên kích thước phần dư.

Khi mô phỏng, phải cập nhật đúng thứ tự và kích thước khoảng trống sau mỗi lần tách; kết quả sau phụ thuộc quyết định trước.



6.5. Phân trang

Phân trang chia chương trình thành các trang và RAM thành các khung cùng kích thước; các trang không cần nằm cạnh nhau.



Trang và khung trang có cùng kích thước

Trang (page)

Một khối cố định trong không gian địa chỉ logic; số trang p dùng để tra bảng trang.

Khung trang (frame)

Một khối cố định trong RAM, bằng kích thước trang; bất kỳ trang nào cũng có thể đặt vào khung trống.

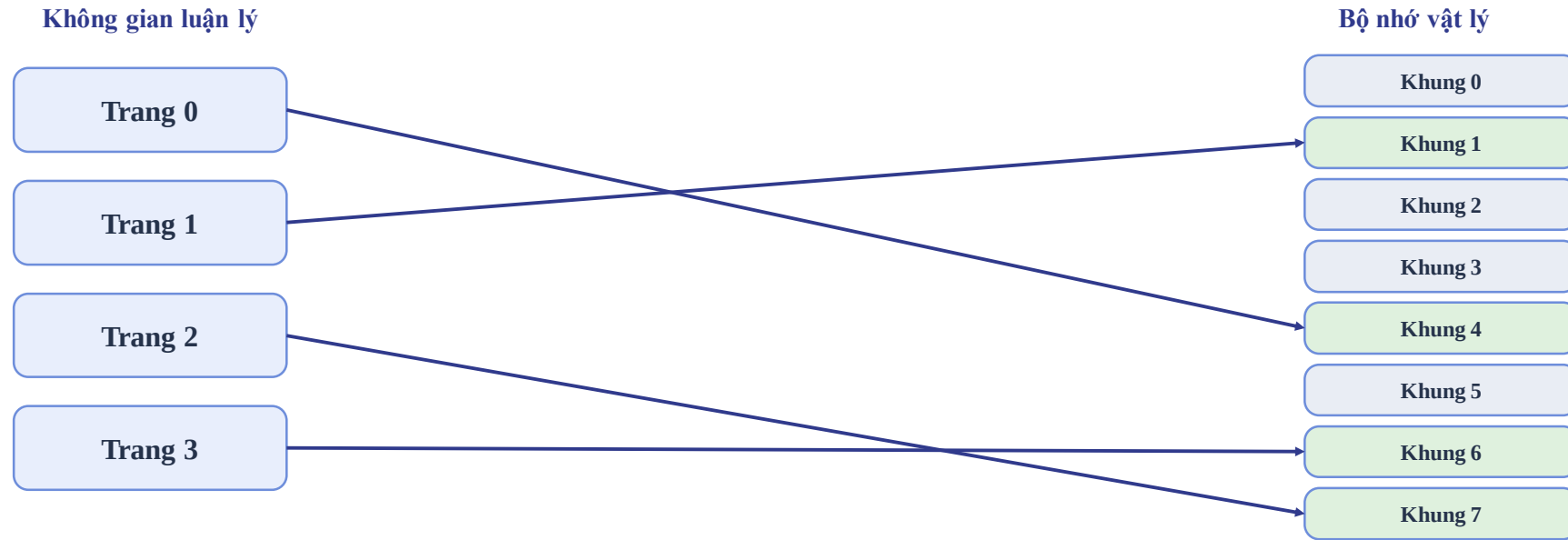
Bảng trang

Ghi trang p đang nằm ở khung nào và lưu các bit quyền, trạng thái.

Kích thước trang thường là lũy thừa của 2 để tách địa chỉ nhanh bằng các bit.



Trang liên tiếp có thể nằm ở khung rời rạc

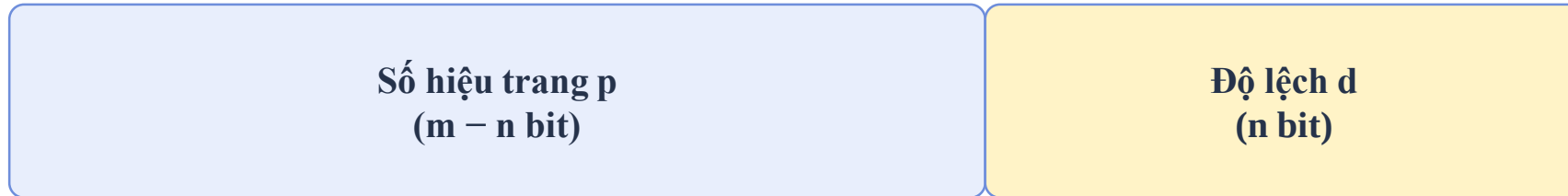


Bảng trang ghi vị trí của từng trang. Vì vậy chương trình vẫn thấy các trang liên tiếp dù chúng nằm ở các khung RAM rời rạc.



Địa chỉ gồm số trang và vị trí bên trong trang

Địa chỉ luận lý m bit



Kích thước trang = 2^n byte; số trang = 2^{m-n}

Số trang p dùng để tra bảng trang. Độ lệch d giữ nguyên vì trang và khung có cùng kích thước.



Tính số bit của địa chỉ

Độ lệch d

Trang có kích thước 2^n byte thì d cần n bit.

Số trang p

Không gian logic 2^m byte có 2^{m-n} trang, nên p cần $m - n$ bit.

Số khung

RAM có 2^f khung thì số hiệu khung cần f bit; địa chỉ vật lý cần $f + n$ bit.

Cần phân biệt số trang của không gian ảo với số khung thật trong RAM.



Ví dụ: 8 trang $\times$ 1 KB ánh xạ vào 32 khung

Địa chỉ luận lý

1 KB = 2^{10} byte nên $d = 10$ bit. Có $8 = 2^3$ trang nên $p = 3$ bit. Tổng: 13 bit.

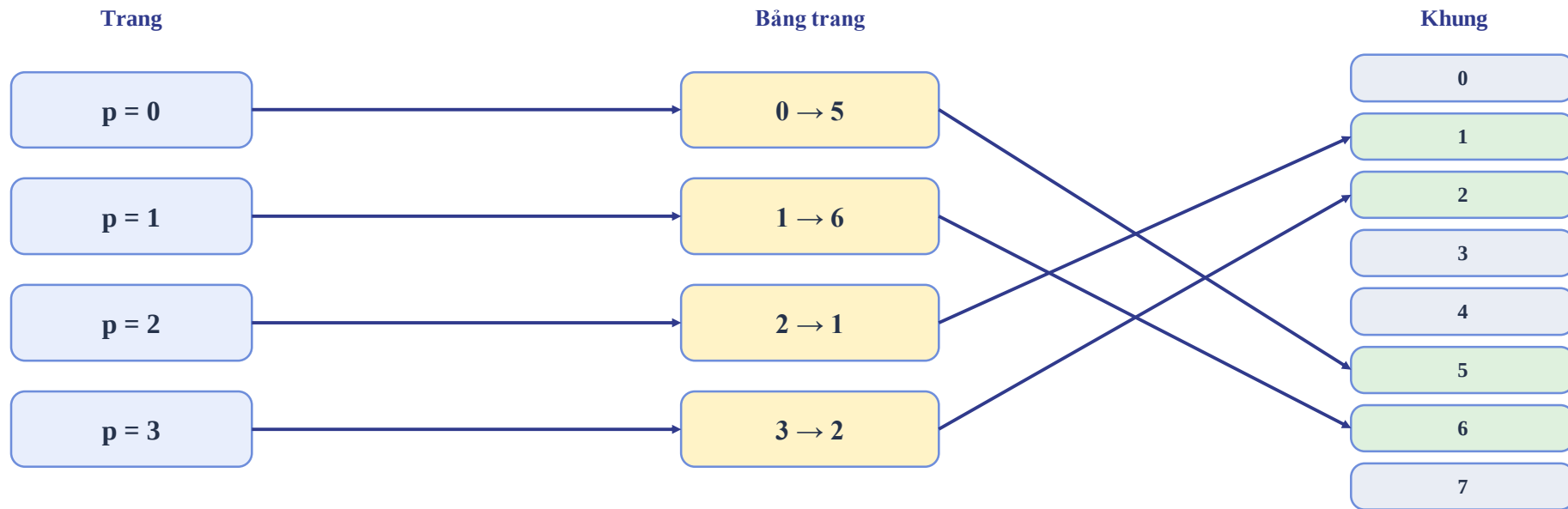
Địa chỉ vật lý

Có $32 = 2^5$ khung nên frame = 5 bit. Ghép với $d = 10$ bit, địa chỉ vật lý cần 15 bit.

Kích thước không gian luận lý = 8 KB; kích thước bộ nhớ vật lý = 32 KB.



Bảng trang biến số trang thành số khung



Ví dụ $p_0 \rightarrow F_5$, $p_1 \rightarrow F_6$, $p_2 \rightarrow F_1$, $p_3 \rightarrow F_2$. Địa chỉ trong trang giữ nguyên độ lệch khi được đặt vào khung tương ứng.



Ví dụ: trang 1 ở khung 6, kích thước trang 4 byte

Từ luận lý sang vật lý

Địa chỉ luận lý 7: $p = \lfloor 7/4 \rfloor = 1$, $d = 7 \bmod 4 = 3$. Tra bảng: frame = 6.

Kết quả

Địa chỉ vật lý = frame $\times$ page_size + d = $6 \times 4 + 3 = 27$.

Phép dịch bảo toàn d; chỉ thay số trang bằng số khung lấy từ bảng trang.



Chọn bất kỳ khung rồi cho trang mới

Trước khi cấp phát

F0	F1	F2	F3
F4	F5	F6	F7
F8	F9	F10	F11

Sau khi cấp 4 trang

F0	F1	F2	F3
F4	F5	F6	F7
F8	F9	F10	F11

Danh sách khung rồi cho phép cấp phát không liên tục. Sau khi chọn F1, F5, F7 và F10, các khung đó được đánh dấu đã dùng và ghi vào bảng trang.



Vì sao tra bảng trang có thể chậm?

PTBR

Thanh ghi trở tới đầu bảng trang của tiến trình đang chạy.

PTLR

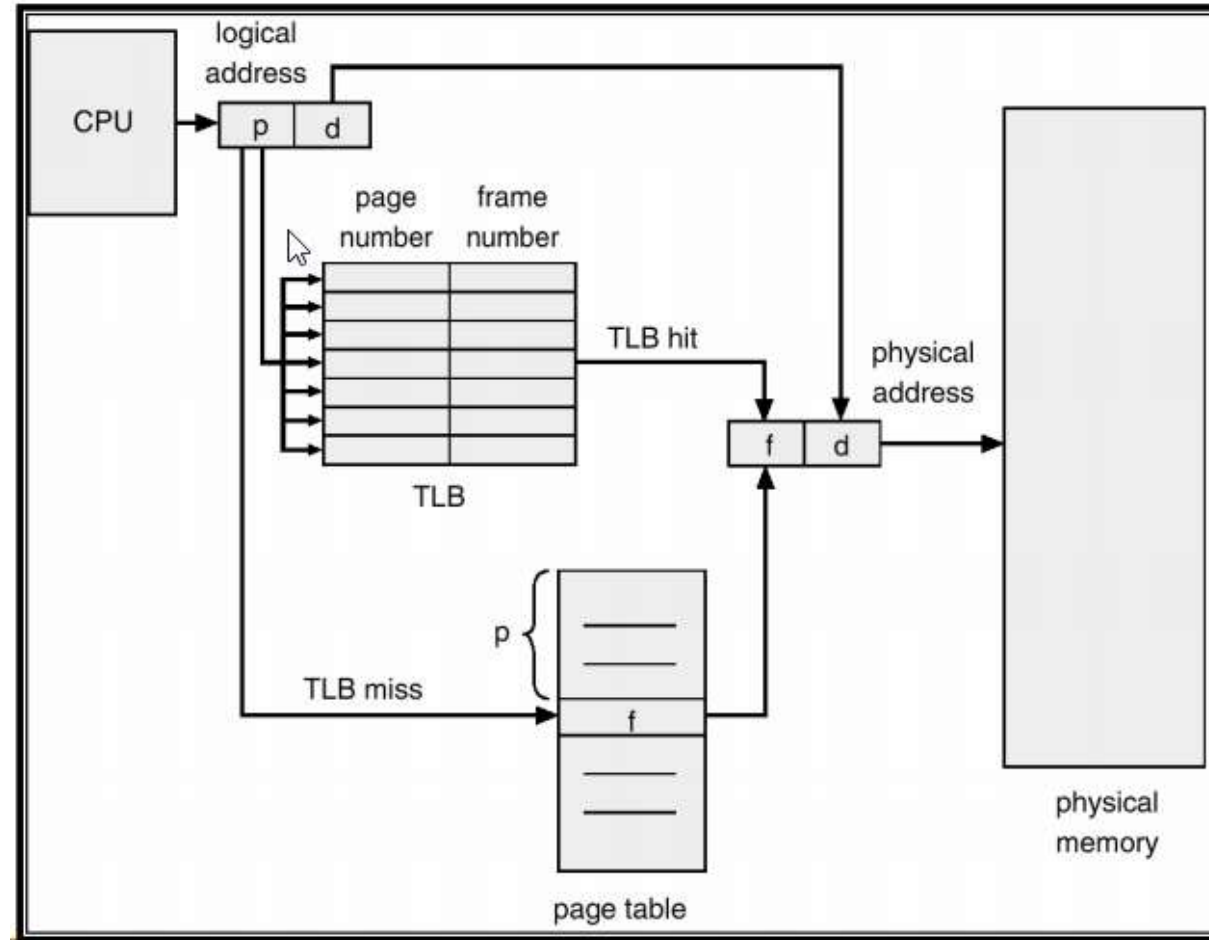
Cho biết số mục hợp lệ hoặc giới hạn của bảng trang.

Hai lần truy cập

Lần 1 đọc bảng trang để tìm khung; lần 2 đọc hoặc ghi dữ liệu thật. TLB giúp giảm bước tra bảng.

Bảng trang lớn có thể được chia nhiều cấp hoặc tổ chức theo kiểu băm/ngược để tiết kiệm bộ nhớ.

TLB: bộ nhớ đệm cho bảng trang



TLB giữ các cặp trang → khung vừa dùng gần đây. Tìm thấy trong TLB thì truy cập nhanh; không thấy thì phải quay về bảng trang.



TLB hit và TLB miss

TLB hit – tìm thấy

Cấp trang → khung đã có trong TLB, nên chỉ cần truy cập dữ liệu trong RAM.

TLB miss – không tìm thấy

Phải đọc bảng trang trong RAM, có thể cập nhật TLB, rồi mới truy cập dữ liệu.

Khi đổi tiến trình, hệ thống có thể xóa TLB hoặc dùng mã định danh để phân biệt các tiến trình.



Các bit trong bảng trang dùng để làm gì?

Hợp lệ / không hợp lệ

Cho biết trang có thuộc vùng nhớ hợp lệ hoặc đang hiện diện hay không; truy cập sai sẽ báo lỗi.

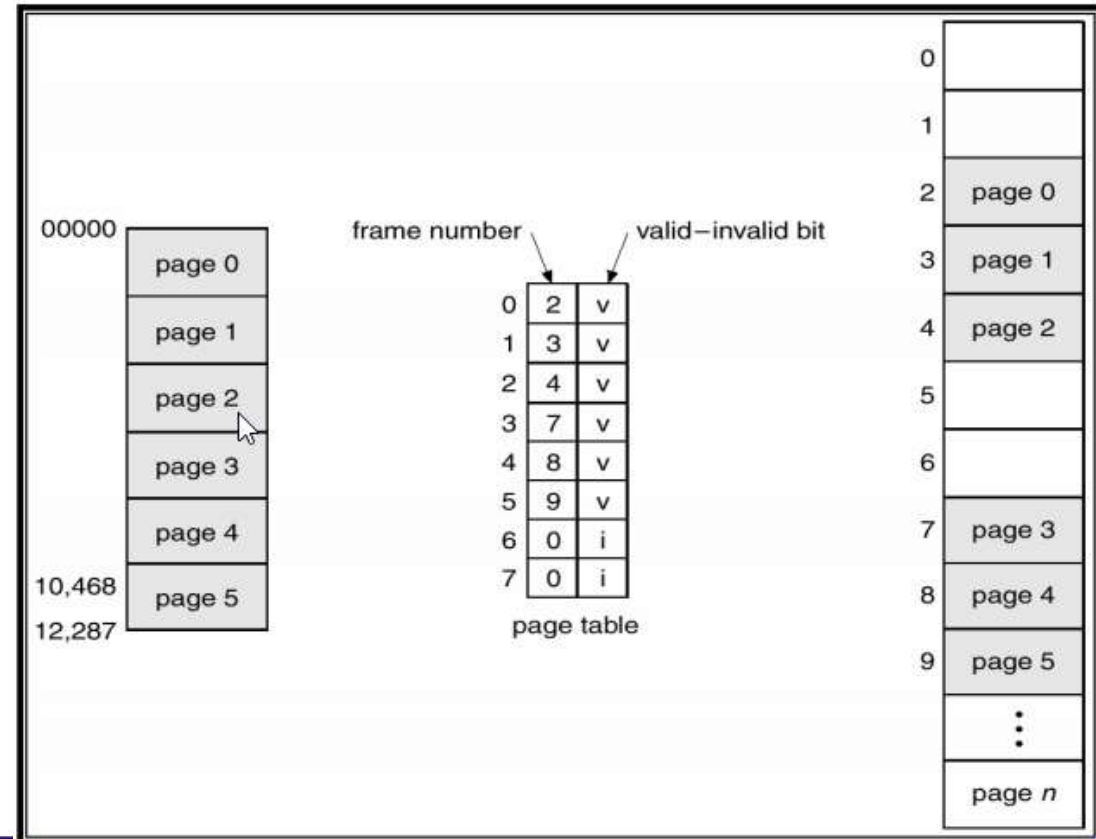
Quyền truy cập

Giới hạn quyền đọc, ghi, thực thi và quyền của người dùng/hệ điều hành.

Theo dõi sử dụng

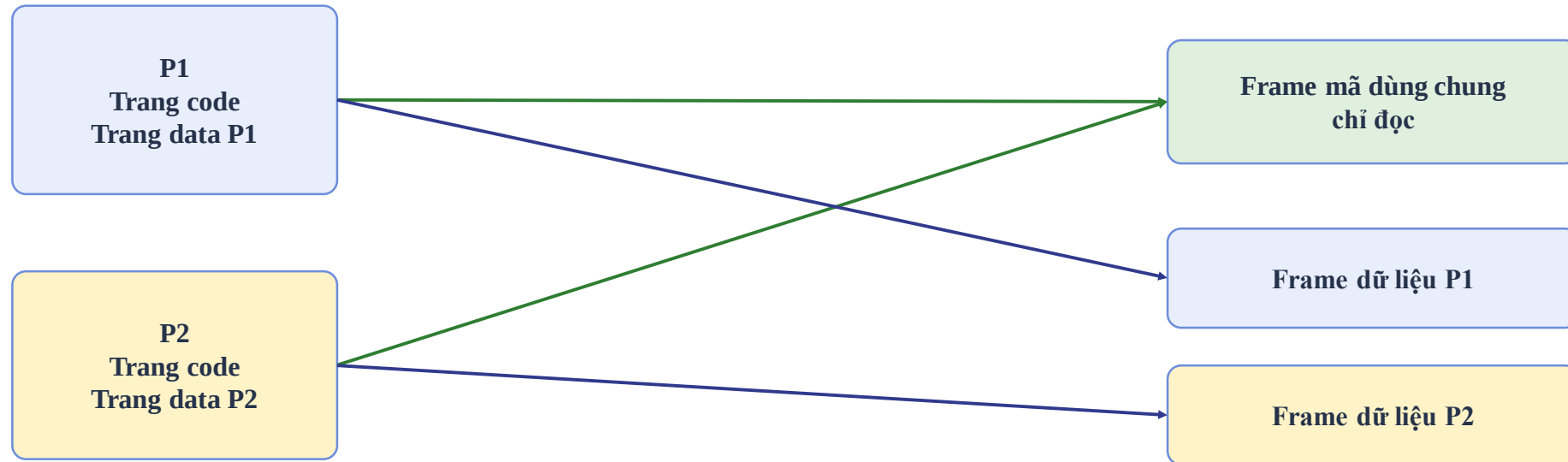
Bit accessed cho biết trang đã được dùng; bit dirty cho biết dữ liệu đã thay đổi và cần ghi lại.

Các bit này giúp hệ điều hành bảo vệ bộ nhớ và chọn trang phù hợp khi cần thay trang.





Mã chỉ đọc có thể được ánh xạ dùng chung



Hai tiến trình có thể cùng trở tới một khung chứa mã chỉ đọc, nhưng vẫn giữ các trang dữ liệu riêng.



Bốn cách tổ chức bảng trang

Một mức

Tra trực tiếp và dễ hiểu, nhưng bảng có thể rất lớn.

Nhiều mức

Chỉ tạo bảng con cho vùng đang dùng; tiết kiệm bộ nhớ nhưng cần tra nhiều bước.

Bảng băm

Dùng số trang ảo để tìm nhóm mục phù hợp; hữu ích với không gian địa chỉ lớn.

Bảng ngược

Mỗi khung RAM có một mục; bảng nhỏ hơn nhưng việc tìm kiếm phức tạp hơn.



Bảng trang nhiều mức tiết kiệm bộ nhớ

Ý tưởng

Chia số trang p thành $p_1, p_2, \dots$; mỗi phần chọn một mức bảng tiếp theo.

Tiết kiệm

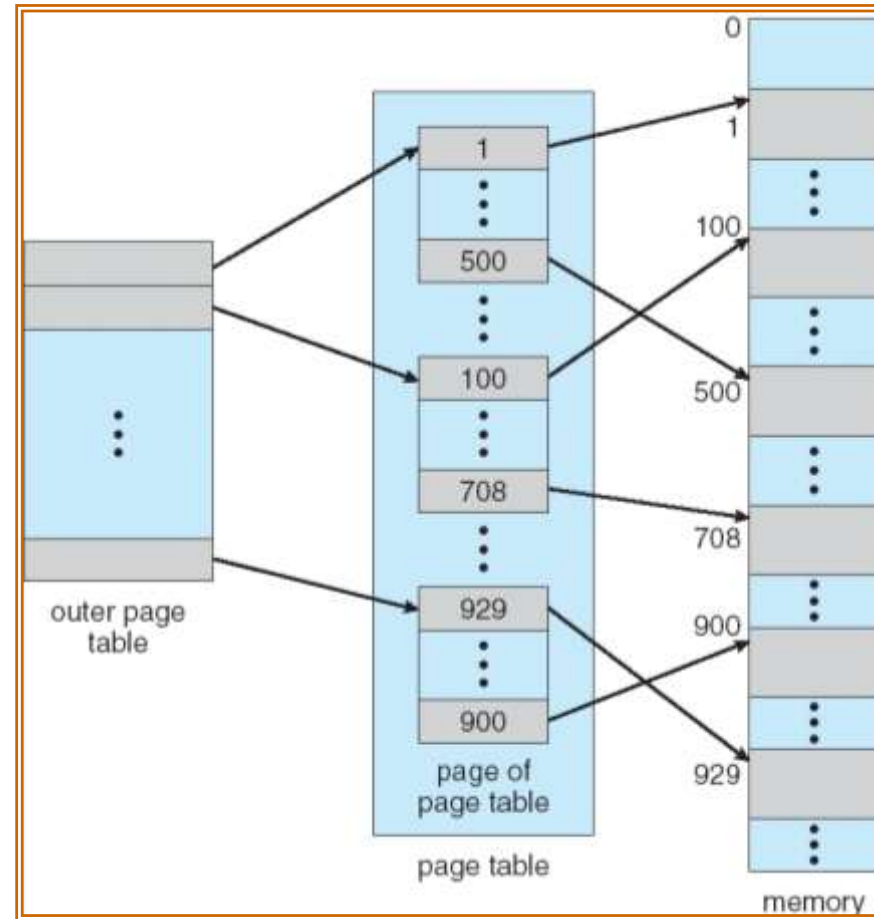
Chỉ tạo bảng con cho các vùng địa chỉ thật sự được sử dụng.

Đánh đổi

Khi TLB không có kết quả, phần cứng phải đọc nhiều mức bảng trước khi tìm được khung.

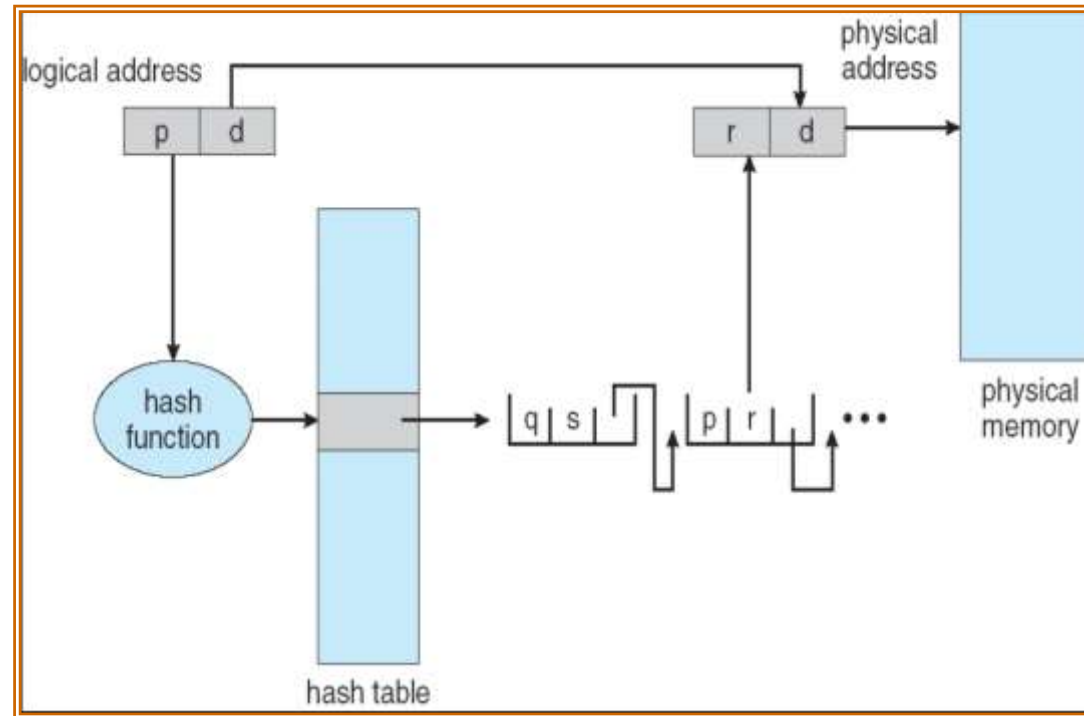
TLB giúp che bớt chi phí của quá trình tra nhiều mức.

Bảng trang hai mức: Tách p thành p_1 và p_2



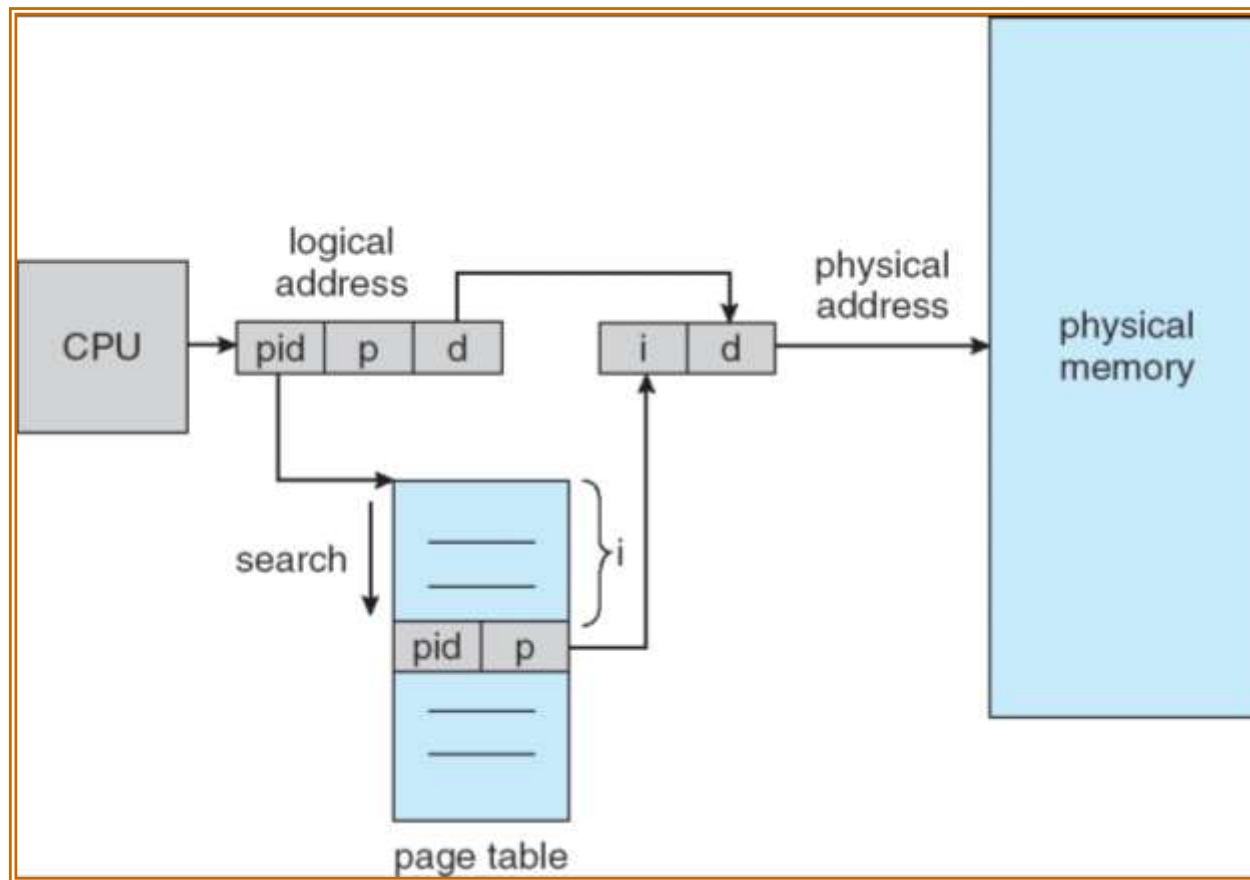
p_1 chọn bảng trang con; p_2 chọn mục trong bảng con để tìm khung. Độ lệch d vẫn giữ nguyên.

Bảng trang băm: Tìm bằng số trang ảo



Số trang ảo (VPN) được đưa qua hàm băm để tìm một nhóm mục. Hệ thống so sánh VPN đầy đủ rồi lấy số khung tương ứng.

Bảng trang ngược: Mỗi khung RAM một mục



Mỗi mục ghi tiến trình nào và trang ảo nào đang nằm trong khung đó. Hàm băm giúp tìm nhanh hơn.



Chọn cấu trúc bảng trang theo nhu cầu

Một mức

Ít bước tra cứu; phù hợp không gian nhỏ nhưng tốn bộ nhớ khi không gian ảo lớn.

Nhiều mức

Phù hợp không gian thừa; tiết kiệm bảng nhưng tăng số bước tra khi TLB miss.

Băm

Phù hợp địa chỉ lớn; tốc độ phụ thuộc hàm băm và số mục va chạm.

Ngược

Kích thước theo số khung RAM; tiết kiệm bảng nhưng khó tìm và chia sẻ hơn.



Phân trang: Lợi ích và chi phí

Không còn phân mảnh ngoài

Mỗi trang có thể đặt vào bất kỳ khung trống nào, nên không cần một vùng RAM liền nhau.

Vẫn có phân mảnh trong

Trang cuối của tiến trình thường không dùng hết.

Chi phí quản lý

Cần bảng trang, TLB và thời gian tra cứu trang → khung.

Trang lớn làm bảng trang nhỏ hơn nhưng có thể tăng phần RAM dư và lượng dữ liệu phải đọc/ghi.



6.6. Phân đoạn

Phân đoạn chia chương trình theo các phần có ý nghĩa như mã, dữ liệu, heap và stack; mỗi phần có kích thước và quyền riêng.



Chương trình gồm nhiều đoạn có ý nghĩa

Mã chương trình

Các hàm và lệnh; thường chỉ cho phép đọc/thực thi và có thể dùng chung.

Dữ liệu

Biến toàn cục, mảng và đối tượng; thường cho phép đọc/ghi.

Heap

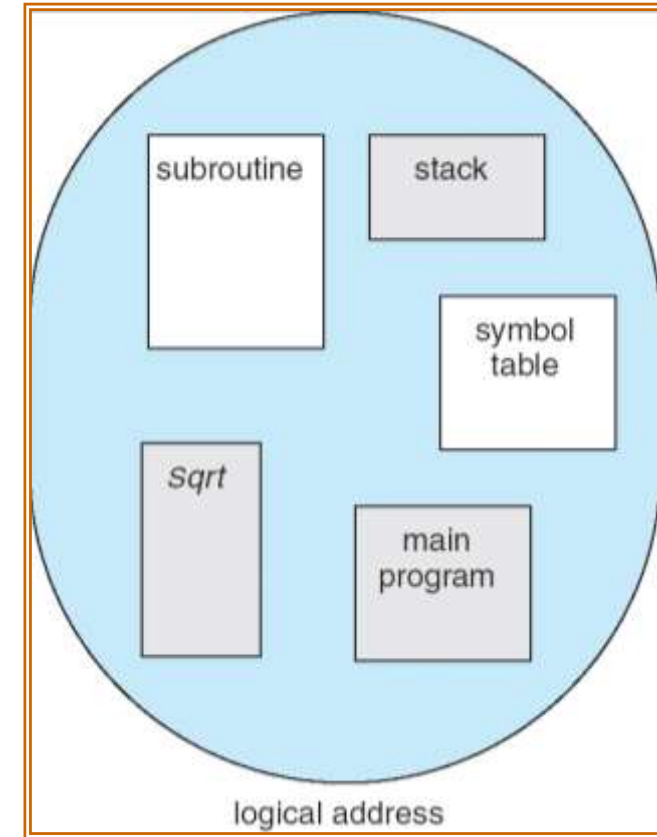
Vùng cấp phát động bằng malloc/new; có thể tăng hoặc giảm khi chương trình chạy.

Stack

Lưu lời gọi hàm, tham số và biến cục bộ; thay đổi theo quá trình gọi hàm.

Phân đoạn gần với cách lập trình viên nhìn chương trình

Chương trình được chia theo ý nghĩa sử dụng



Mỗi đoạn có số hiệu, vị trí, kích thước và quyền riêng. Không gian địa chỉ là một tập các đoạn, không phải một dải byte duy nhất.



Địa chỉ phân đoạn gồm số đoạn và độ lệch

Số đoạn s

Chọn mục s trong bảng đoạn để lấy Base, Limit và quyền truy cập.

Độ lệch d

Vị trí bên trong đoạn. Nếu $d < \text{Limit}[s]$, địa chỉ vật lý = $\text{Base}[s] + d$.

Mỗi đoạn có kích thước khác nhau nên giới hạn của d cũng khác nhau.



Quyền truy cập được đặt riêng cho từng đoạn

Hợp lệ

Cho biết đoạn có tồn tại trong không gian của tiến trình hay không.

Đọc / Ghi / Thực thi

Mã thường chỉ đọc/thực thi; dữ liệu đọc/ghi; stack thường không cho thực thi.

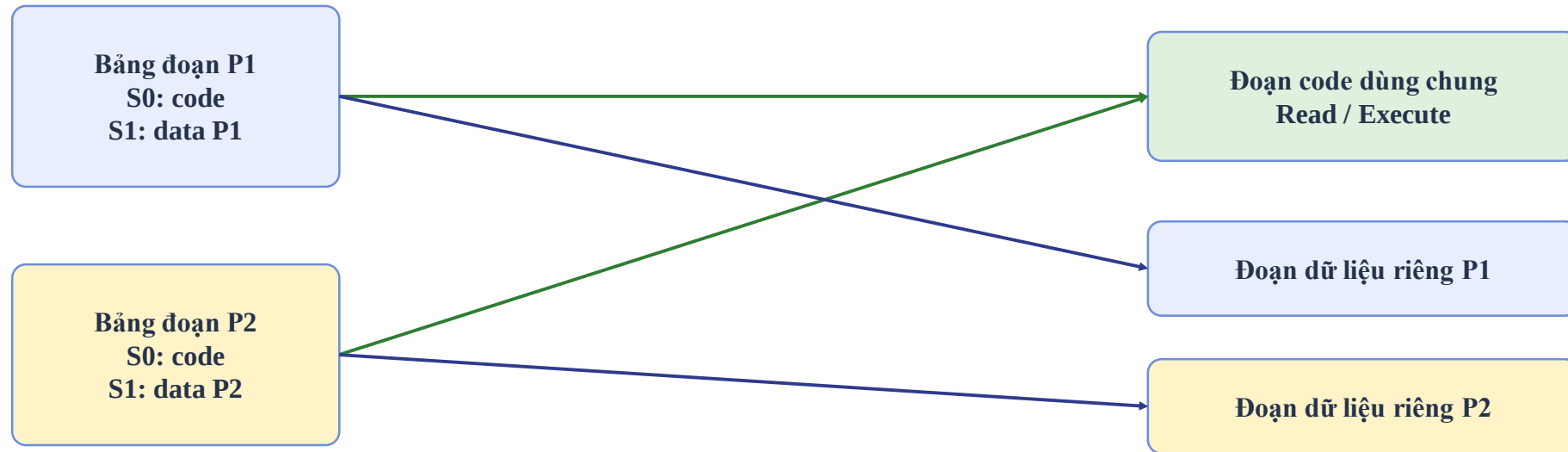
Chia sẻ

Hai tiến trình có thể cùng trỏ tới một đoạn mã thư viện nhưng giữ dữ liệu riêng.

Phân đoạn để đặt quyền theo ý nghĩa, nhưng kích thước thay đổi có thể gây phân mảnh ngoài.



Chia sẻ đoạn mã, giữ dữ liệu riêng



Hai tiến trình dùng chung đoạn mã chỉ đọc, còn đoạn dữ liệu của mỗi tiến trình nằm ở vùng riêng.



Phân đoạn và phân trang khác nhau thế nào?

Phân đoạn

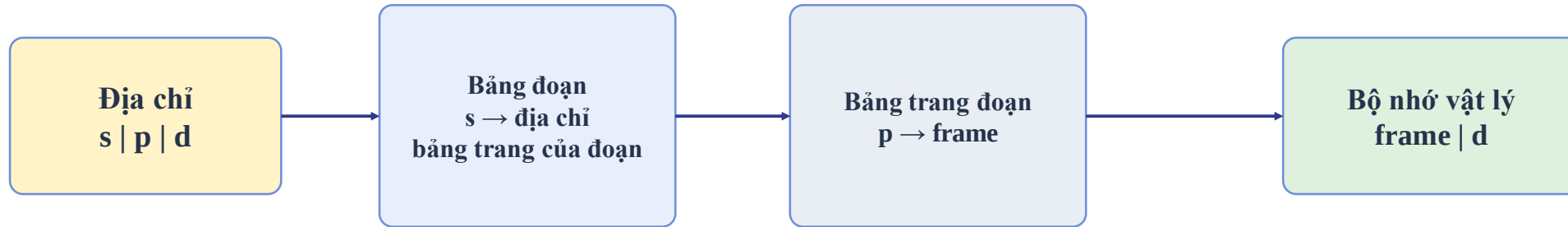
Chia theo ý nghĩa, kích thước thay đổi, dễ bảo vệ/chia sẻ; có thể gây phân mảnh ngoài.

Phân trang

Chia thành khối cố định, chương trình không cần biết; không có phân mảnh ngoài nhưng có phân mảnh trong.

Có thể kết hợp: chia chương trình thành đoạn, sau đó phân trang từng đoạn.

Kết hợp phân đoạn và phân trang



s chọn đoạn; trong đoạn, p chọn khung qua bảng trang; d chọn byte trong khung. Cách này giữ ý nghĩa của đoạn và tránh phải cấp một vùng vật lý liền nhau.

Tổng kết Chương 6



Chương trình dùng địa chỉ logic; MMU dịch sang địa chỉ vật lý và kiểm tra quyền.

Swapping tạm đưa tiến trình ra ổ đĩa; Overlay chỉ nạp mô-đun đang cần vào vùng nhớ dùng chung.

Cấp phát liên tục dễ hiểu nhưng gặp phân mảnh ngoài; First-fit, Best-fit và Worst-fit chọn vùng trống theo cách khác nhau.

Phân trang dùng trang, khung và bảng trang; TLB giúp tra cứu nhanh hơn.

Phân đoạn chia chương trình theo mã, dữ liệu, heap và stack; có thể kết hợp với phân trang.



Câu hỏi và bài tập thảo luận

1. Mô phỏng First-fit và Best-fit cho các vùng trống 100, 500, 200, 300, 600 KB với yêu cầu 212, 417, 112, 426 KB.

2. Xét một không gian địa chỉ có 8 trang, mỗi trang có kích thước 1Kbyte. Ánh xạ vào bộ nhớ vật lý có 32 khung trang.

a) Địa chỉ luận lý (logical address) gồm bao nhiêu bit?

b) Địa chỉ vật lý (physical address) gồm bao nhiêu bit?

3. Cho bảng phân trang (bảng ánh xạ) của một process như hình, hãy cho biết:

a. Địa chỉ vật lý 6578 sẽ được chuyển thành địa chỉ luận lý bao nhiêu? Biết rằng kích thước

mỗi frame là 1 KB.

B. Địa chỉ luận lý 3654 sẽ được chuyển thành địa chỉ vật lý bao nhiêu? Biết rằng kích thước

mỗi frame là 2 KB

0	6
1	4
2	5
3	7
4	1
5	9

Bảng trang của P1

Email: minhnl@utc.edu.vn

Tổng kết Chương 7



Chỉ giữ phần đang dùng trong RAM; phần còn lại nằm trên ổ đĩa.

Trang chưa có gây lỗi trang; hệ điều hành nạp trang rồi chạy lại lệnh FIFO đơn giản; OPT là chuẩn tốt nhất;

LRU bám sát lịch sử sử dụng.

Phải cân bằng số khung, tập làm việc và tần suất lỗi trang.

Xảy ra khi các tiến trình không có đủ khung cho vùng đang hoạt động.

Nạp từng đoạn logic khi cần; linh hoạt nhưng khó tìm vùng liên tục



Câu hỏi và bài tập thảo luận

Vì sao chỉ một tỷ lệ lỗi trang rất nhỏ cũng làm hệ thống chậm rõ rệt?

Mô phỏng FIFO, OPT và LRU với chuỗi: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5; dùng 3 khung.

Dấu hiệu nào cho thấy hệ thống đang thrashing? Nêu hai cách xử lý.

Phân biệt lỗi trang và lỗi đoạn.

Vì sao kết hợp phân đoạn với phân trang giúp giảm phân mảnh ngoài?

Email: minhnl@utc.edu.vn